

Retrieval Augmented Generation

- What is RAG?

⇒ RAG is an architecture to help LLMs to provide a better & response by using relevant information from additional resources. RAG is a technique that enhances language model generation by incorporating external knowledge.

- Why use RAG?

- ⇒ 1. Limited Knowledge Access
2. Hallucinations

- RAG Architecture

- ⇒ 1. Ingestion
2. Retrieval
3. Generation

- Ingestion

⇒ Document → Extract data → chunking
↓
Build Semantic Index ← Creating embeddings
↓
Knowledge Base (Vector/Graph DB)

- Retrieval

⇒ Question → Query Embeddings
↓
Semantic search to Knowledgebase

- Generation

⇒ Response received from retrieval + Prompt → LLM call → LLM response

- What is Embedding

⇒ Numerical representation of data

- What is Corpus

⇒ The complete data/document is called Corpus

- Why chunking is required

⇒ Sending the corpus to LLM may overload it, and may lead to hallucination, that's why we chunk the corpus and send the relevant chunks.

- Deciding factors for chunk size

- ⇒ 1. Data characteristics
2. Retrieve constraints
3. Memory and computational resources
4. Task requirements
5. Experimentations
6. Overlap considerations

• Type of Embeddings

⇒ Frequency based Embeddings

Neural Network based Embeddings

→ BOW

→ TF-IDF

→ N-GRAMS

ଅବନୀ ଧାରା

ଅବନୀ ଧାରା

→ Word2Vec

→ FastText

→ Bert

→ Elmo

→ OpenAI Embeddings

→ Gemini Embeddings

} Embedding generate

ଅବନୀ ଧାରା text embedding models ଅବନୀ ଧାରା

• Similarity Search mechanisms

⇒ Dot product → $A \cdot B$

$$\text{Cosine similarity} \rightarrow \cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \cdot B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

■ RAG Implementation with LangChain:

• Libraries required

- ⇒
1. langchain-community → for useful wrapper functions
 2. langchain
 3. langchain-text-splitters → for chunking
 4. langchain-huggingface → for huggingface inferencing
 5. langchain-chroma → for useful wrappers for chroma db.
 6. Chroma → vector database
 7. langchain-groq → for Groq interfacing

• Ingestion Part

⇒ `loader = PyPDFLoader("documents/compliance.pdf")`
`docs = loader.load()`

document ଫାଇଲ ଅଟେ pdf load

`text-splitter = RecursiveCharacterTextSplitter(chunk_size=220, chunk_overlap=40)`

`final_chunks = text-splitter.split_documents(docs)`

document to split
 ଅବନୀ, ଅବନୀ chunk
 maximum 220 ବା word
 ଅବନୀ, 40 ବା word overlap
 ଅବନୀ

```
embedding-model = HuggingFaceEndpointEmbeddings(
    huggingfacehub-api-token = os.environ["HF_TOKEN"],
    model="sentence-transformers/all-MiniLM-L6-v2",
    task="feature-extraction",
)
```

→ "all-MiniLM-L6-v2" model use vector embedding create chunk documents

```
vector-store = Chroma.from_documents(
    documents=final_chunks, embedding=embedding-model,
    persist_directory=".chroma-db"
)
```

→ document and embedding create the chroma db and store vector, for later run-time use.

```
vector-store = Chroma(
    persist_directory=".chroma-db",
    embedding_function=embedding-model
)
```

→ Retrieval of data, for use later.

• Retrieval Part

⇒ query = "What is the main topic of the PDF?"

```
docs_mapping = vector-store.similarity-search(query, K=5)
```

→ query string find similarity search result, top 5 results return.

```
context = "\n--\n".join([doc.page-content for doc in docs_mapping])
```

→ top 5 results as string return.

- Generation Part

⇒ system-prompt = (

"You are a helpful assistant. Answer the user's question using ONLY the provided text context."

"If the answer cannot be found in the context, say 'I cannot find the answer in the document.'"

"Do not make up information or use outside knowledge."

)

user-prompt = { "Context:

{ context }

Question: { query }

Answer: ""

→ Seeding the context in the user prompt

```
llm = ChatGPTQ(model="openai/gpt-oss-120b", temperature=0,  
               max_tokens=None)
```

```
messages = [{"system", system-prompt}, {"user", user-prompt}]
```

```
response = llm.invoke(messages).content
```

```
print(response)
```

HNSW (Hierarchical Navigable Small World)

HNSW is a graph-based ANN (Approximate Nearest Neighbour) algorithm. It is universally considered the fastest and most reliable option for RAG applications where query latency and high accuracy are paramount.

It is a multi-layered graph where the top layers have sparse connections and bottom layers have dense connections.

- Speed $\rightarrow O(\log N)$
- Accuracy $\rightarrow 95-99\%$

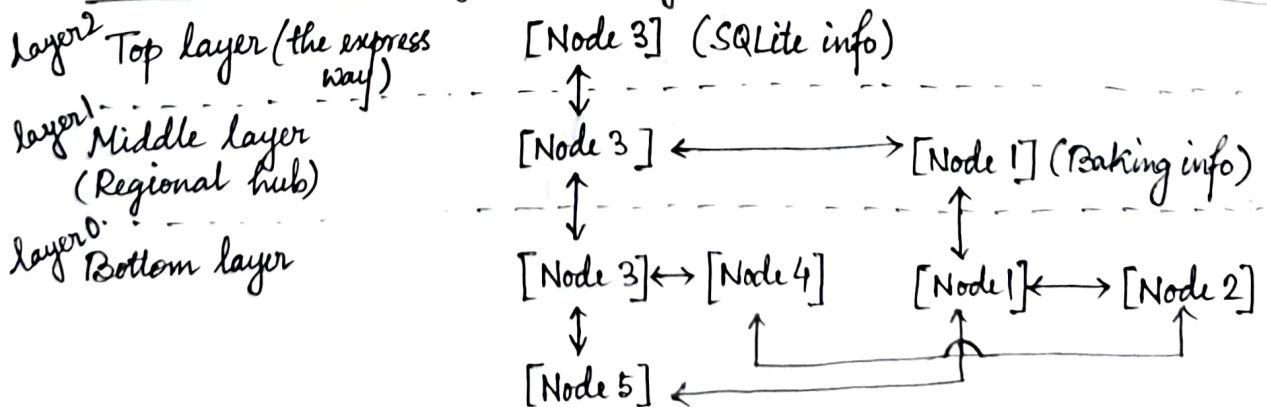
■ How the graph is structured?

- $\rightarrow$ 1. Each node inserted becomes a node in the graph
Random Probability distribution $\rightarrow$ assign each node to layer λ
2. Every node keeps track of its own list of neighbours.

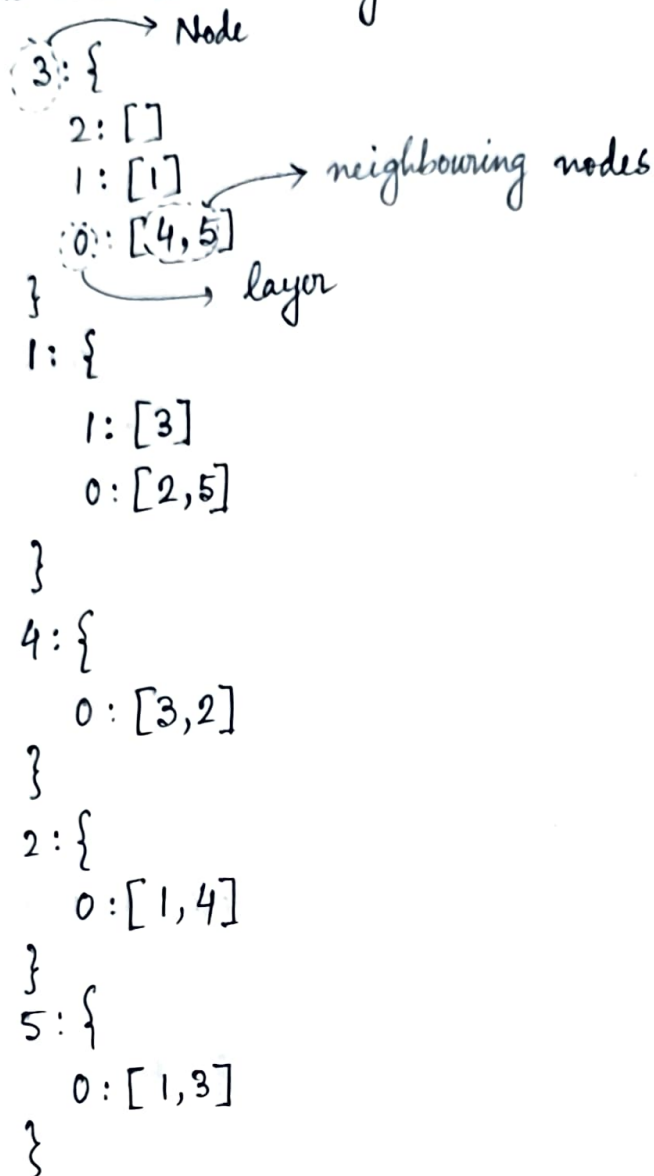
■ Graph visualization

$\rightarrow$ Let's say we have 5 text

- ID1 : How to bake sourdough bread
- ID2 : Baking dynamic croissants from scratch
- ID3 : How to optimize SQLite database queries
- ID4 : Writing efficient manual indexes in python
- ID5 : French pastry and baking techniques



The node structure may look like this:



• Scaling factor

→ This determines exactly how many nodes are allowed to climb up to the higher layers, ensuring the graph maintains a perfect multi-layered skip-list structure.

Main goal → Exponential decay → પ્રતિ નોડ, તોપ લેયર અર્થે એક નોડ હોવો જોઈએ

- Layer 0: 100% node હોવો જોઈએ
- Layer 1: અંદાજે 10% node હોવો જોઈએ
- Layer 2: અંદાજે 1% node હોવો જોઈએ
- Layer 3: અંદાજે 0.1% node હોવો જોઈએ

The formula used to assign a random layer to a new node:

$$\text{layer} = \lfloor -\ln(\text{random}()) \times m_L \rfloor \quad \lfloor \Rightarrow \text{floor symbol} \rfloor$$

if we choose $m_L = \frac{1}{\ln(M)}$ the math works out perfectly where $M \Rightarrow$ the maximum number of connections a node can have.

Lets say maximum connection $M = 16$

then, $m_L = 1.0 / \ln(16) \approx 0.3606$

Now layer assignment,

Random draw(r)	$-\ln(r)$	$-\ln(r) \times m_L$	layer $\lfloor -\ln(r) \times m_L \rfloor$
0.75	$-\ln(0.75) = 0.287$	$0.287 \times 0.3606 = 0.103$	$\lfloor 0.103 \rfloor = \boxed{0}$
0.30	$-\ln(0.30) = 1.203$	$1.203 \times 0.3606 = 0.434$	$\lfloor 0.434 \rfloor = \boxed{0}$
0.05	$-\ln(0.05) = 2.995$	$2.995 \times 0.3606 = 1.08$	$\lfloor 1.08 \rfloor = \boxed{1}$
0.001	$-\ln(0.001) = 6.907$	$6.907 \times 0.3606 = 2.49$	$\lfloor 2.49 \rfloor = \boxed{2}$

Manual HNSW Class

Parameters →

- dim (dimension)
- M (maximum number of connection one node can have)
- M0 (maximum number of connection one node can have at 0th layer)
- ef_construction (Search depth at building stage)
- ef_search (Search depth during query phase)
- ml (level normalization factor)
- nodes (the actual graph that holds the connections between layers and nodes)

- enter_node } Tracks the global entryway details of
- max_layer } the whole highway system.
- conn (SQLite connection)
- total_nodes (Number of all nodes present in the graph)

■ Cosine distance

→ Formula ⇒ $1 - \text{cosine similarity (between a query embedding and chunk embeddings)}$

cosine distance $\Rightarrow$ $\frac{\text{distance between two nodes}}{\text{distance between two nodes}}$

■ Important Algorithms

- -search_layer
- Greedily travels across a single layer of HNSW graph to find the closest local neighbor to the query vector

INPUT:

- ~~- conn: the active SQLite database connection~~
- query_vector: query embedding array of float32
- enter_node: the integer SQLite id where we begin the search
- layer(0): the current integer layer number we are traversing.

OUTPUT:

The integer SQLite id of the local node closest to the query

BEGIN:

SET curr_node = enter_node

SET curr_dist = cosine distance between query vector and curr_node

LOOP INFINITELY:

SET changed = False

SET neighbours = get neighbours of curr_node for layer l

For each neighbour in neighbours:

SET neighbour_dist = cosine distance between query vector and neighbour

if neighbour_dist < curr_dist:

SET curr_dist = neighbour_dist

SET curr_node = neighbour

SET changed = True

END if

END For

if not changed:

BREAK

END if

END LOOP INFINITY

RETURN curr_node

END

Explanation:

1. Get the cosine distance of the enter_node
2. In an infinite loop:
get the current node, get all its neighbours from the same layer.
check the cosine distance between query vector and every neighbour, and save details of the node if the distance is lesser than the previous one.
3. Break the loop if • no lesser distance is found
4. Return the node id.

- - search-layer-ef
- Bounded BEAM search in a layer for greedy multicandidate node search.

INPUT:

q : Query vector

enter-node: starting node id

l : current layer index

ef: size limit of the tracking pool.

OUTPUT:

A list of upto 'ef' node ids closest to query vector q

BEGIN:

SET $v = \{\text{enter-node}\}$ // Set containing all visited node IDs

SET $\text{dist} = \text{cosine distance between query vector and enter-node}$

INITIALIZE C Min-priority queue of candidates to explore

INITIALIZE W Max-priority queue to store the closest nodes found, sorted by distance

INSERT (enter-node, dist) TO C & W

WHILE C is not empty:

SORT C & W by distance

SET curr-node, curr-dist = POP front from C

SET worst-node, worst-dist = ~~POP~~ PEEK back from W

IF curr-dist > worst-dist:

BREAK

END IF

SET neighbours = get neighbours of curr-node of layer l

FOR each neighbour in neighbours:

IF v doesn't contain neighbour :

INSERT neighbour to v

SET neighbour-dist = cosine distance between query vector
and neighbour

SET worst_node, worst_dist = PEEK back of W

IF neighbour-dist < worst_dist OR Size of W < ef :

INSERT (neighbour, neighbour-dist) to C

INSERT (neighbour, neighbour-dist) to W

IF Size of W > ef :

DROP the furthest tracked element from W

END IF

END IF

END IF

END FOR

END WHILE

RETURN all node ids from W

END

Explanation :

When traversing the graph, the algorithm constantly evaluates unvisited neighbours, add them to the closest nodes max-priority queue and kick-out the worst performer to stay inside the limits.

- insert

→ This method inserts a new node in the HNSW graph to proper layer

INPUT:

new-node-id : The unique integer id for the incoming node

new-vector : The raw embedding vector float array for the chunk

OUTPUT:

The internal graph network structure updated with bidirectional links

BEGIN:

SET self.total-nodes += 1

// Case 1: Graph is completely empty

IF self.nodes is empty:

SET (

self.enter-node = new-node-id

self.nodes.new-node-id = {0:[]}

self.max-layer = 0

)

RETURN

END IF

// Case 2: Graph is not empty

SET insert-layer = random layer

SET self.nodes.new-node-id = empty object

FOR i, 0 to insert-layer:

self.nodes.new-node-id.i = empty list

END FOR

SET curr_obj = self.enter-node

END FOR

END FOR

END IF

END

3. if the random layer is greater than max layer $\rightarrow$ set random layer as the max layer, and new node as the enter-node.

- search

→ This method traverse the graph as quickly as possible to reach the ground layer, then do precise checking for the closest chunks.

INPUT:

query: The raw text string from user prompt

K: The number of top relevant document chunks to return

OUTPUT:

An array of text chunks sorted by semantic relevance

BEGIN:

SET query-vector = get text embeddings for query

IF self.total-nodes < 10000:

do brute-force similarity search and return the chunks

END IF

IF self.enter-node or self.nodes is empty:

RETURN empty list

END IF

// Phase 1: High-speed zoom down to layer 1

FOR l from self.max-layer down to 1:

SET curr-obj = self.-search-layer(query-vector, curr-obj, l)

END For

// Phase 2: Ground floor exploration and structural Traversal checks

SET candidates = self.-search-layer-ef(query-vector,
curr-obj, 0, self.ef-search)

SET top-K-ids = self.get-top-K(query-vector, candidates, K)

RETURN list of chunks related to top-K-ids

END

Explanation:

1. High-speed zoom down to layer 1 from top layer
2. Get all the relevant vectors that align with query vector
3. Sort the retrieved chunks based on relevance and return top K chunks as an array.